Praxi Physics Lab

Class Diagram — Plain-Language Guide

A clear walkthrough of every major box and arrow in
`architecture/Praxi_Class_Diagram.mmd`

How to read the Mermaid file · What each layer does · Who owns whom · Runtime path

1. How to read the diagram

The file `Praxi_Class_Diagram.mmd` is a UML-style class diagram. Open it on `mermaid.live`, GitHub, or a Mermaid VS Code preview. This PDF explains the same structure in words so you can walk a reviewer through it without squinting at boxes.

1.1 Arrow meanings

Line style (concept)	Means	Example in this project
Solid triangle (inheritance)	“Is a” / implements or extends	PendulumExperiment implements Experiment
Solid line with label	Owns or uses at runtime	Host owns MeasurementRecorder
Dashed line	Depends on / calls / builds	main injects scene adapter into Host
Module stereotype	Not a class — a file of functions	PendulumEquations, Primitives

1.2 Mental model (one sentence)

main starts a fixed-timestep **loop** that drives **ExperimentHost**, which loads an **Experiment** from the **registry**; that experiment uses **physics** for math and **rendering** for Three.js; the **UI** only talks to the store, host, and measurement snapshots — never to a specific experiment type.

2. The five layers (top to bottom)

Layer	Folder	Job in one line
Bootstrap	src/main.ts	Wires everything; injects rendering into core
UI	src/ui/	Schema controls, graph, scalars, compare toggle
Core runtime	src/core/	Host, registry, loop, params, comparison (no Three.js)
Experiments	src/experiments/	Glue: physics + visuals + measurements
Physics / Rendering	src/physics/ · src/rendering/	Pure math · Three.js scene & meshes

Hard rule: physics never imports Three.js. core never imports Three.js. Experiments are the only place that mixes both. UI never hardcodes per-experiment controls.

3. The Experiment contract (center of the diagram)

In the Mermaid file, four small interfaces feed into one big **Experiment<C>**. That is Interface Segregation in code: reviewers can see the slices, but you still implement **one** type (what the assessment asks for).

3.1 The four slices

Interface	Methods	Plain meaning
Parameterized	getParameterSchema, setParameters	Tell the UI which sliders to build; accept new values
Steppable	update(dt), reset	Advance physics by a fixed dt; restore initial state
Measurable	getMeasurements	Hand graphs/scalars a snapshot of time-series + readouts
SceneAttached<C>	setup, dispose, render?	Create/destroy 3D objects; optional smooth visual lerp

3.2 What is C?

C is the setup context type. In this project it is **ExperimentRenderContext** (scene, root, camera, renderKit, recorder, optional syncCameraTarget). Core defines `Experiment<C>` without knowing Three.js; rendering owns the concrete

context. That is Dependency Inversion.

3.3 Supporting data shapes (also on the diagram)

- **ParameterSchema** — one slider/toggle definition (key, label, min/max/step, default).
- **MeasurementChannel** — one plotted series (id, label, unit, Float64Array values).
- **ScalarMetric** — one number card (value, optional theoretical + % error).
- **MeasurementSnapshot** — time + channels + scalars + count (+ comparison metadata).
- **ExperimentRegistration** — { id, name, factory } stored in the registry.
- **ExperimentSceneAdapter** — how Host creates/disposes the live 3D context without importing Three.
- **Integrator** — { step(state, derivatives, dt) } for RK4 / SemiImplicitEuler.

4. Core runtime boxes

4.1 ExperimentHost

The thin conductor. It does **not** contain physics equations or Three.js calls. It switches experiments, steps them, asks for measurements, and forwards comparison to ComparisonController.

- **switchExperiment(id)** — dispose old visuals → factory from registry → adapter.prepare → setup(context) → schema → reset.
- **fixedUpdate(dt)** — current.update(dt) + comparison.fixedUpdate(dt).
- **render(alpha)** — optional visual interpolation between fixed steps.
- **getMeasurements()** — primary snapshot, optionally merged with B for the graph.
- **setComparisonEnabled / setComparisonEditTarget** — UI compare controls.

4.2 ComparisonController

Owns parameter sets **A** and **B**. When compare is on, it clones the same experiment type into a **headless** (offscreen) context — never drawn — and keeps stepping it. Panel edits go to A or B based on Edit target. Graph gets B channels renamed with suffix __**B**.

4.3 ParameterStore · MeasurementRecorder · SimulationLoop · Registry

Box	Owns	Does
ParameterStore	Current slider values	Notifies Host/comparison when UI changes a value
MeasurementRecorder	Ring buffers for time + channels	Experiments append samples; UI reads snapshots
SimulationLoop	RAF + accumulator	Physics at FIXED_DT=1/120; render gets alpha
ExperimentRegistry	Map of id → factory	registerExperiment / getExperiment / listExperiments

mergeComparisonSnapshots (function on the diagram) copies A channels and adds B channels with __B ids so GraphSystem can overlay them.

5. Physics boxes

Two integrator classes implement **Integrator**. Equation “modules” are files of pure functions (shown as <<module>> on the diagram).

Piece	Used by	Exactly what it does
RK4	Pendulum, Spring	4th-order Runge–Kutta — stable for oscillators
SemiImplicitEuler	Projectile	Update velocity then position — good for trajectories with drag
PendulumEquations	PendulumExperiment	State [θ,ω]; θ"=-(g/L)sinθ−cω; energy; small-angle period
SpringEquations	SpringExperiment	State [x,v]; x"=-(k/m)x-(c/m)v; energy; undamped frequency
ProjectileEquations	ProjectileExperiment	State [x,y,vx,vy]; gravity+drag; analytic no-drag path; range
PeriodAnalysis	All three	measurePeriod / measureFrequency / percentDifference

6. Rendering boxes

Box	Plain meaning
ExperimentRenderContext	The bag passed to setup(): scene, root, camera, renderKit, recorder, syncCameraTarget?
SceneManager	Long-lived WebGL world: lights, sky, ground, OrbitControls, render()
PrimaryExperimentSceneAdapter	Host’s live-viewport helper: prepare env, build context, dispose meshes

OffscreenFactory	Builds a throwaway Three scene for comparison B (never shown)
RenderKit	Tracks geometries/materials so disposeAll() prevents GPU leaks on switch
Primitives	createSphere/Rod/Line/Spring/Box/Marker + in-place line/spring updates
CameraUtils	frameCameraToBounds — fit camera to projectile path; sync orbit target

On the diagram: **PrimaryExperimentSceneAdapter** implements **ExperimentSceneAdapter**. Host only sees the interface; main.ts constructs the concrete adapter and offscreen factory.

7. Experiment classes (the three boxes that implement Experiment)

Each class implements the full Experiment surface. Arrows to RK4/Euler, equation modules, and Primitives show glue — not framework knowledge of which experiment is active.

Class	Integrator	What you see / measure
PendulumExperiment	RK4	Rod+bob; angle/ω/energy channels; period vs $2\pi\sqrt{(L/g)}$
SpringExperiment	RK4	Helical spring + mass; displacement/velocity/energy; frequency vs theory
ProjectileExperiment	Semi-implicit Euler	Cannon trail; predicted dashed path; range predicted vs actual; launched toggle

Registration: only `experiments/index.ts` calls `registerExperiment({ id, name, factory })`. Adding a fourth experiment = new file + one registry entry (and usually one physics equations file).

8. UI boxes

Class	Talks to	Does
ParameterPanel	ParameterStore	Builds controls from schema only
GraphSystem	MeasurementSnapshot	Plots channels; Energy mode; A/B dashed overlays; CSV
ScalarDisplay	ScalarMetric[]	Theory cards with % error
UIUpdateScheduler	Scalar + Graph	Throttles DOM updates ~15 Hz
TopBar	Host callbacks	Compare A/B, Edit A/B, Reset
ExperimentSwitcher	Host + Registry	Dropdown of listExperiments()
ResultsPanelResize	CSS + Graph	Drag results width; notify graph to reflow

UI primitives (`createPanel/Slider/Toggle/…`) are not drawn as classes in the Mermaid file — they are tiny factories with no experiment knowledge.

9. Who owns whom (follow the solid arrows)

```
ExperimentHost
  ■■ ParameterStore          (listens for UI edits)
  ■■ MeasurementRecorder     (shared with active experiment)
  ■■ ComparisonController
  ■    ■■ Experiment B?      (optional headless clone)
  ■■ ExperimentSceneAdapter  (injected — live 3D)
  ■■ Experiment A            (current factory instance)

SimulationLoop ■■calls■■■ Host.fixedUpdate / Host.render
main ■■injects■■■ Adapter + OffscreenFactory into Host
```

10. One frame, end to end

- **1.** SimulationLoop accumulates time; while enough time remains, Host.fixedUpdate(FIXED_DT).
- **2.** Experiment A.update: copy prev state → integrator.step → recorder.append.
- **3.** If compare on, Experiment B.update the same way (offscreen).
- **4.** Host.render(alpha): experiments lerp visuals between prev and current state.
- **5.** SceneManager.render draws the live scene (orbit controls update).
- **6.** UIUpdateScheduler may refresh graph/scalars from Host.getMeasurements() (merged A+B).

If the user moves a slider: ParameterStore notifies → ComparisonController.applyPanelParams → setParameters on A or B → usually reset. If they switch experiments: Host.dispose + new factory + new schema.

11. How to present this diagram in a review

- Start at **Experiment** and the four ISP slices — “one contract, schema UI, measurements.”
- Point to **Host** as orchestration with **no Three.js** in core.
- Show **Adapter / OffscreenFactory** as DIP: rendering injected from main.
- Show one experiment arrow into **RK4 + equations + primitives** — glue layer.
- Show **UI** → **Store / Host / Snapshot** — no per-experiment UI.
- End with “new experiment = one file + registry entry.”

12. Companion files

- `architecture/Praxi_Class_Diagram.mmd` — the Mermaid source of the diagram.
- `src/core/types.ts` — live contracts (always prefer code if diagram drifts).
- `src/experiments/README.md` — how to add an experiment.
- `README.md` — run instructions and architecture summary.

> If the diagram and the code ever disagree, **trust the TypeScript**. Update the .mmd when contracts change.